



MySQL - RDBMS

Trainer: Ms. Nisha Dingare



MySQL Programming

- RDBMS Programming is an ISO standard – part of SQL standard – since 1992.
- SQL/PSM stands for Persistent Stored Module.
- Inspired from PL/SQL - Programming language of Oracle.
- PSM allows writing programs for RDBMS. The program contains set of SQL statements along with programming constructs e.g. variables, if-else, loops, case, ...
- PSM is a block language. Blocks can be nested into another block.
- MySQL program can be a stored procedure, function or trigger.



MySQL Programming

- MySQL PSM program is written by db user (programmers).
- It is submitted from client, server check syntax & store them into db in compiled form.
- The program can be executed by db user when needed.
- Since programs are stored on server in compiled form, their execution is very fast.
- All these programs will run in server memory.



Stored Procedure

- Stored Procedure is a routine. It contains multiple SQL statements along with programming constructs.
- Procedure doesn't return any value (like void fns in C).
- Procedures can take zero or more parameters.
- Procedures are created using CREATE PROCEDURE and deleted using DROP PROCEDURE.
- Procedures are invoked/called using CALL statement.
- Result of stored procedure can be
 - returned via OUT parameter.
 - inserted into another table.
 - produced using SELECT statement (at end of SP).
- Delimiter should be set before writing procedure.



Stored Procedure

```
CREATE TABLE result(v1 DOUBLE, v2 VARCHAR(50));
```

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_hello()
```

```
BEGIN
```

```
    INSERT INTO result VALUES(1, 'Hello World');
```

```
END;
```

```
$$
```

```
DELIMITER ;
```

```
CALL sp_hello();
```

```
SELECT * FROM result;
```

```
-- 01_hello.sql (using editor)
```

```
DROP PROCEDURE IF EXISTS sp_hello;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_hello()
```

```
BEGIN
```

```
    SELECT 1 AS v1, 'Hello World' AS v2;
```

```
END;
```

```
$$
```

```
DELIMITER ;
```

```
SOURCE /path/to/01_hello.sql
```

```
CALL sp_hello();
```



Stored Procedure – PSM Syntax

VARIABLES

```
DECLARE varname DATATYPE;  
DECLARE varname DATATYPE DEFAULT init_value;  
SET varname = new_value;  
SELECT new_value INTO varname;  
SELECT expr_or_col INTO varname FROM table_name;
```

PARAMETERS

```
CREATE PROCEDURE sp_name(PARAMTYPE p1 DATATYPE)  
BEGIN  
    ...  
END;  
  
-- IN param: Initialized by calling program.  
-- OUT param: Initialized by called procedure.  
-- INOUT param: Initialized by calling program and  
-- modified by called procedure  
-- OUT & INOUT param declared as session variables.
```

```
CREATE PROCEDURE sp_name(OUT p1 INT)  
BEGIN  
    SELECT 1 INTO p1;  
END;  
  
SET @res = 0;  
CALL sp_name(@res);  
SELECT @res;
```

IF-ELSE

```
IF condition THEN  
    body;  
END IF;  
-----  
IF condition THEN  
    if-body;  
ELSE  
    else-body;  
END IF;  
-----  
IF condition THEN  
    if1-body;  
ELSE  
    IF condition THEN  
        if2-body;  
    ELSE  
        else2-body;  
    END IF;  
END IF;  
-----  
IF condition THEN  
    if1-body;  
ELSEIF condition THEN  
    if2-body;  
ELSE  
    else-body;  
END IF;
```

LOOPS

```
WHILE condition DO  
    body;  
END WHILE;  
-----  
REPEAT  
    body;  
UNTIL condition  
END REPEAT;  
-----  
label: LOOP  
IF condition THEN  
    ...  
    LEAVE label;  
END IF;  
...  
END LOOP;  
  
WHEN condition THEN  
    body;  
WHEN condition THEN  
    body;  
ELSE  
    body;  
END CASE;
```

SHOW PROCEDURE

```
SHOW PROCEDURE STATUS  
LIKE 'sp_name';  
  
SHOW CREATE PROCEDURE sp_name;
```

DROP PROCEDURE

```
DROP PROCEDURE  
IF EXISTS sp_name;
```



MySQL Exceptions

- Exceptions are runtime problems, which may arise during execution of stored procedure, function or trigger.
- Required actions should be taken against these errors.
- SP execution may be continued or stopped after handling exception.
- MySQL error handlers are declared as:
 - DECLARE *action* HANDLER FOR *condition* *handler_impl*;
- The *action* can be: CONTINUE or EXIT.
- The *condition* can be:
 - MySQL error code: e.g. 1062 for duplicate entry.
 - SQLSTATE value: e.g. 23000 for duplicate entry, NOTFOUND for end-of-cursor.
 - Named condition: e.g. DECLARE duplicate_entry CONDITION FOR 1062;
- The *handler_impl* can be: Single liner or PSM block i.e. BEGIN ... END;



MySQL Stored Functions

- Stored Functions are MySQL programs like stored procedures.
- Functions can be having zero or more parameters. MySQL allows only IN params.
- Functions must return some value using RETURN statement.
- Function entire code is stored in system table.
- Like procedures, functions allows statements like local variable declarations, if-else, case, loops, etc. One function can invoke another function/procedure and vice-versa. The functions can also be recursive.
- There are two types of functions: DETERMINISTIC and NOT DETERMINISTIC.

CREATE FUNCTION

```
CREATE FUNCTION fn_name(p1 TYPE)
RETURNS TYPE
[NOT] DETERMINISTIC
BEGIN
    body;
    RETURN value;
END;
```

SHOW FUNCTION

```
SHOW FUNCTION STATUS LIKE 'fn_name';

SHOW CREATE FUNCTION fn_name;
```

DROP FUNCTION

```
DROP FUNCTION IF EXISTS fn_name;
```



MySQL Triggers

- Triggers are supported by all standard RDBMS like Oracle, MySQL, etc.
- Triggers are not supported by WEAK RDBMS like MS-- Access.
- Triggers are not called by client's directly, so they don't have args & return value.
- Trigger execution is caused by DML operations on database.
 - BEFORE/AFTER INSERT, BEFORE/AFTER UPDATE, BEFORE/AFTER DELETE.
- Like SP/FN, Triggers may contain SQL statements with programming constructs. They may also call other SP or FN.
- However COMMIT/ROLLBACK is not allowed in triggers. They are executed in same transaction in which DML query is executed.

CREATE TRIGGER

```
CREATE TRIGGER trig_name
AFTER|BEFORE dml_op ON table
FOR EACH ROW
BEGIN
    body;
    -- use OLD & NEW keywords
    -- to access old/new rows.
    -- INSERT triggers - NEW rows.
    -- DELETE triggers - OLD rows.
END;
```

SHOW TRIGGERS

```
SHOW TRIGGERS FROM db_name;
```

DROP TRIGGER

```
DROP TRIGGER trig_name;
```



MySQL Triggers

- Applications of triggers:
 - Maintain logs of DML operations (Audit Trails).
 - Data cleansing before insert or update data into table. (Modify NEW value).
 - Copying each record AFTER INSERT into another table to maintain "Shadow table".
 - Copying each record AFTER DELETE into another table to maintain "History table".
 - Auto operations of related tables using cascading triggers.
- Cascading triggers
 - One trigger causes execution of 2nd trigger, 2nd trigger causes execution of 3rd trigger and so on.
 - In MySQL, there is no upper limit on number of levels of cascading.
 - This is helpful in complicated business processes.
- Mutating table error
 - If cascading trigger causes one of the earlier trigger to re-execute, "mutating table" error is raised.
 - This prevents infinite loop and also rollback the current transaction.





- Thank you!

